

10-Day Java Full Stack Roadmap for Saquib

Goal: Master remaining full-stack technologies + build portfolio projects in 10 days

Schedule: ~10-12 hours/day (Learning: 5-6 hrs | Coding/Projects: 5-6 hrs)

Your Current Foundation

- Core Java, OOP - DONE
 - MySQL, Oracle SQL - DONE
 - JDBC - DONE
 - HTML, CSS - DONE
 - Servlets - DONE
 - Git - DONE
 - JavaScript (Basics) - DONE
 - DSA - In Progress
-

Day 1 - Advanced Java + JavaScript Deep Dive

Theme: Strengthen Java & JS fundamentals before frameworks

Morning (5-6 hrs) - Advanced Java

- [] Collections Framework deep dive (ArrayList, HashMap, LinkedList, TreeMap, HashSet)
- [] Generics - bounded types, wildcards (<? extends>, <? super>)
- [] Exception Handling - custom exceptions, try-with-resources
- [] Java 8 Features - Lambda expressions, Functional interfaces
- [] Streams API - filter, map, reduce, collect, forEach
- [] Optional class - avoiding NullPointerException

Lecture Videos - Advanced Java

>> [Telusko - Java Tutorial for Beginners \(Full Playlist\)](#)

https://www.youtube.com/playlist?list=PLsyebzWxl7pe_liTfNyr55kwJPWbgxB5

>> [Telusko - Lambda Expressions in Java 8](#)

<https://www.youtube.com/watch?v=SEy1sC9sM1w>

>> [Telusko - Stream API in Java](#)

<https://www.youtube.com/watch?v=J95H25p600A>

>> [Kunal Kushwaha - Complete Java + DSA Course \(Your existing resource\)](#)

https://youtube.com/playlist?list=PL9gnSGHScnr_DxHsP7AW9ftq0AtAyYqJ

Afternoon (5-6 hrs) - JavaScript Essentials

- [] ES6+ Features - let/const, arrow functions, template literals, destructuring
- [] Array methods - map, filter, reduce, forEach, find, some, every
- [] Promises & Async/Await - fetch API, error handling
- [] DOM Manipulation - querySelector, event listeners, dynamic content

[] JSON - parse, stringify, working with API responses

Lecture Videos - JavaScript

>> [Chai aur Code \(Hitesh Choudhary\) - Complete JavaScript Series \(Hindi\)](#)

<https://www.youtube.com/watch?v=Hr5iLG7sUa0&list=PLu71SKxNbfoBuX3f4EOACle2y-tRC5Q37>

>> [Namaste JavaScript by Akshay Saini - Deep JS Concepts](#)

<https://www.youtube.com/watch?v=pN6jk0uUrD8&list=PLlasXeu85E9cQ32gLCvAvr9vNaUccPVNP>

>> [SuperSimpleDev - JavaScript Full Course \(Beginner to Pro\)](#)

<https://www.youtube.com/watch?v=EerdGm-ehJQ>

TIP

Java 8 features (Lambdas, Streams) are heavily asked in interviews and used everywhere in Spring Boot. Master them well.

Day 2 - Maven + Spring Core + Spring Boot Basics

Theme: Enter the Spring ecosystem

Morning (5-6 hrs) - Build Tools & Spring Fundamentals

[] Maven - pom.xml, dependencies, lifecycle (clean, install, package)

[] Spring Core: IoC, Dependency Injection (Constructor, Setter, Field)

[] ApplicationContext & Bean lifecycle

[] @Component, @Autowired, @Configuration, @Bean

[] Spring Boot Introduction: Why Spring Boot over Spring?

[] Spring Initializr - create your first project (start.spring.io)

[] Project structure, application.properties

[] @SpringBootApplication, auto-configuration

Afternoon (5-6 hrs) - First Spring Boot App

[] @RestController, @RequestMapping, @GetMapping, @PostMapping

[] @PathVariable, @RequestParam, @RequestBody

[] ResponseEntity & HTTP status codes

[] Build: Student Management REST API (in-memory, no DB yet)

[] Test with Postman (install if not already)

Lecture Videos - Spring Boot

>> [Telusko - Spring Boot & Spring Framework Tutorial with Project](#)

<https://www.youtube.com/watch?v=vlz9ina4Usk>

>> [Telusko - Spring 6 and Spring Boot Playlist](#)

<https://www.youtube.com/playlist?list=PLsyeobzWxl7o3-T0F8-Qp-wQ-v5-oB06Z>

>> [Java Guides - Spring Boot Tutorial for Beginners Playlist](#)

https://www.youtube.com/playlist?list=PLGRDMO4rOGcNSBOJOIrgQqGpIgo6_VZgR

>> [Telusko - Spring Framework & Microservices Full Course \(15+ hrs\)](#)

<https://www.youtube.com/watch?v=XyP9H9s2k4g>

Day 3 - Spring Data JPA + Hibernate + MySQL Integration

Theme: Connect Spring Boot to your MySQL database

Morning (5-6 hrs) - JPA & Hibernate

- [] ORM Concepts - what is ORM, why JPA over JDBC
- [] JPA Annotations: @Entity, @Table, @Id, @GeneratedValue, @Column
- [] Hibernate - Session, SessionFactory, HQL basics
- [] Spring Data JPA: JpaRepository, custom query methods, @Query
- [] Database Config - application.properties for MySQL

Afternoon (5-6 hrs) - Upgrade Project to JPA

- [] Refactor Student Management API to use MySQL + JPA
- [] Layered Architecture: Controller -> Service -> Repository -> Database
- [] Entity Relationships: @OneToMany, @ManyToOne, @ManyToMany
- [] Validation: @Valid, @NotNull, @Size, @Email

Lecture Videos - JPA & Hibernate

>> [Java Guides - Spring Boot Full Course \(5 hrs, includes JPA + MySQL\)](#)

<https://www.youtube.com/watch?v=msXL1C8L2x4>

>> [Java Guides - Spring Boot Tutorial Playlist](#)

https://www.youtube.com/playlist?list=PLGRDMO4rOGcNSBOJOIrgQqGpIgo6_VZgR

>> [Baeldung - Spring Tutorials \(Written Reference\)](#)

<https://www.baeldung.com/>

>> [Spring Official Docs](#)

<https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/>

Day 4 - Exception Handling + Logging + Service Layer Patterns

Theme: Write production-quality backend code

Morning (5-6 hrs) - Robust Backend

- [] Global Exception Handling: @ControllerAdvice + @ExceptionHandler
- [] Custom exception classes (ResourceNotFoundException, etc.)
- [] Logging: SLF4J + Logback (@Slf4j from Lombok)
- [] Lombok: @Data, @Getter, @Setter, @Builder, @NoArgsConstructor
- [] DTO Pattern: Entity <-> DTO mapping (ModelMapper or MapStruct)
- [] Service Layer: @Service, @Transactional

Afternoon - Start Project 1: Employee Management System

- [] Tech Stack: Spring Boot + JPA + MySQL + REST APIs
- [] Employee CRUD (name, email, department, salary)
- [] Department CRUD with @OneToMany relationship
- [] Search/filter employees by department

- [] Pagination & Sorting (Pageable, Sort)
- [] Global exception handling + DTO layer + validation

IMPORTANT

Project 1 (Employee Management System) will be a core portfolio piece. Make the code clean, well-structured, and push to GitHub.

Day 5 - Spring Security + JWT Authentication

Theme: Secure your APIs

Morning (5-6 hrs) - Security Fundamentals

- [] Authentication vs Authorization
- [] SecurityFilterChain, UserDetailsService
- [] Password encoding with BCrypt
- [] Role-based access (@PreAuthorize, hasRole())
- [] JWT: Header, Payload, Signature
- [] JWT flow: Login -> Token -> Authenticated requests
- [] JWT utility class (generate, validate, extract)
- [] JWT Authentication Filter

Afternoon - Add Auth to Project 1

- [] /api/auth/register - create user with encrypted password
- [] /api/auth/login - return JWT token
- [] Protect Employee endpoints - require valid JWT
- [] Role-based: ADMIN -> full CRUD, USER -> read-only

Lecture Videos - Spring Security + JWT

>> [Amigoscode - Spring Security Tutorial \(2023\)](https://www.youtube.com/watch?v=Jm9TMZi7V4)

<https://www.youtube.com/watch?v=Jm9TMZi7V4>

>> [Amigoscode - Spring Boot + Spring Security with JWT \(Access & Refresh Tokens\)](https://www.youtube.com/watch?v=kYJ4C4pC_Jg)

https://www.youtube.com/watch?v=kYJ4C4pC_Jg

>> [Java Guides - Spring Security 6 with Spring Boot 3 Crash Course](https://www.youtube.com/watch?v=R23554YmN_Y)

https://www.youtube.com/watch?v=R23554YmN_Y

>> [Java Brains - Spring Security + JWT from scratch](https://www.youtube.com/watch?v=x2JUmH1yJ9w)

<https://www.youtube.com/watch?v=x2JUmH1yJ9w>

Day 6 - React.js Fundamentals

Theme: Enter the modern frontend world

Morning (5-6 hrs) - React Core

- [] Setup: Node.js, npm, Vite (npm create vite@latest)
- [] JSX - syntax, expressions, conditional rendering
- [] Components - functional components, props, component tree

- [] State - useState hook, state updates, re-rendering
- [] Event Handling - onClick, onChange, onSubmit
- [] Lists & Keys - rendering arrays with .map()
- [] Conditional Rendering - ternary, &&, if/else in JSX

Afternoon (5-6 hrs) - More React + Practice

- [] useEffect Hook - side effects, API calls, dependency array
- [] React Router - BrowserRouter, Routes, Route, Link, useNavigate
- [] Styling - CSS Modules or Tailwind CSS
- [] Build: Multi-page React app with routing, counter, API fetch

Lecture Videos - React.js

>> [Chai aur Code \(Hitesh Choudhary\) - Complete React Course \(Hindi\)](#)

<https://www.youtube.com/watch?v=vz1RIUyrc3w&list=PLu71SKxNbfoDqgPchmvlsL4hTnJlrtige>

>> [Codevolution - ReactJS Tutorial for Beginners Playlist](#)

<https://www.youtube.com/playlist?list=PLC3y8-rFNvxueEb1hadH0RSuLYm4G-9tK>

>> [React Official Documentation \(Best written resource\)](#)

<https://react.dev/learn>

TIP

Use Vite instead of Create React App -- it is significantly faster and is now the recommended approach.

Day 7 - React Advanced + Axios + Connecting to Backend

Theme: Connect React frontend to Spring Boot backend

Morning (5-6 hrs) - Advanced React

- [] Forms in React - controlled components, form validation
- [] Axios - GET/POST/PUT/DELETE, interceptors, base URL config
- [] Context API - createContext, useContext, sharing state globally
- [] Custom Hooks - reusable logic (e.g., useFetch)

Afternoon - Start Project 2 Frontend: Employee Management UI

- [] Connect React to Spring Boot Employee API (Day 4-5)
- [] CORS Configuration in Spring Boot
- [] Employee List, Add/Edit Employee forms
- [] Login & Register pages
- [] Store JWT in localStorage, attach to Axios headers
- [] Protected Routes - redirect to login if no token

Lecture Videos - Full-Stack (React + Spring Boot)

>> [Java Guides - ReactJS + Spring Boot CRUD Full Stack App Playlist](#)

<https://www.youtube.com/playlist?list=PLGRDMO4rOGcNLnW1L2vgsExTBg-VPoZHr>

>> [Java Guides Blog - Spring Boot + React Full Stack Written Tutorial](#)

<https://www.javaguides.net/2023/03/spring-boot-react-full-stack-tutorial.html>

Day 8 - Complete Full-Stack Project 2 + Polish

Theme: Finish and polish your full-stack application

- ☐ Department management pages (CRUD)
- ☐ Search & Filter functionality in the UI
- ☐ Pagination component for employee list
- ☐ Toast notifications (react-toastify)
- ☐ Loading spinners during API calls
- ☐ Responsive design - make it mobile-friendly
- ☐ Clean folder structure (components, pages, services, context, hooks, utils)
- ☐ README.md - project description, tech stack, screenshots
- ☐ Push to GitHub - both frontend and backend repos
- ☐ Test entire flow: Register -> Login -> CRUD -> Logout

IMPORTANT

Project 2 (Full-Stack Employee Management) is your FLAGSHIP portfolio project. Add screenshots to README, write clean commit messages.

Day 9 - Project 3: Blog/Notes Application (Speed Build)

Theme: Build a complete project from scratch to prove speed & skill

Backend (Spring Boot)

- ☐ User registration/login with JWT
- ☐ Post CRUD - title, content, category, created date
- ☐ Category CRUD
- ☐ User-post relationship (@ManyToOne)
- ☐ Pagination & sorting on posts
- ☐ Search posts by title/keyword

Frontend (React)

- ☐ Home page - list all posts with pagination
- ☐ Single post view
- ☐ Create/Edit post
- ☐ Category filter sidebar
- ☐ Auth - login/register + protected routes
- ☐ Responsive navbar

TIP

This speed-build tests your ability to build end-to-end from scratch -- exactly what interviewers want to see. Time yourself!

Day 10 - Deployment + Portfolio + Interview Prep

Theme: Ship it, showcase it, prepare for interviews

Morning - Deployment & Portfolio

- [] Deploy Backend: Railway.app or Render (free tier)
- [] Deploy Frontend: Vercel or Netlify (free)
- [] Deploy Database: PlanetScale or Railway MySQL
- [] Update LinkedIn with projects + tech stack
- [] Add GitHub links with clean READMEs

Deployment Links

>> [Railway.app - Deploy Spring Boot \(Free\)](#)

<https://railway.app>

>> [Vercel - Deploy React Apps \(Free\)](#)

<https://vercel.com>

>> [Render - Deploy Backend \(Free\)](#)

<https://render.com>

>> [Netlify - Deploy Frontend \(Free\)](#)

<https://www.netlify.com>

Afternoon - Interview Preparation Topics

- [] Java: OOP, Collections, Multithreading, String immutability, == vs .equals(), Java 8
 - [] Spring Boot: IoC, DI, Bean lifecycle, @Component vs @Service vs @Repository
 - [] Spring Security: JWT flow, SecurityFilterChain
 - [] React: Virtual DOM, useState vs useEffect, Props vs State
 - [] SQL: Joins, Group By, Having, Subqueries, Indexing, Normalization
 - [] DSA: Revise all solved problems
-

Tools & Extras

Git & GitHub

>> [Kunal Kushwaha - Complete Git & GitHub Tutorial](#)

<https://www.youtube.com/watch?v=apGV9Kg7ics>

>> [Traversy Media - Git & GitHub Crash Course](#)

<https://www.youtube.com/watch?v=HUBNt18RFbo>

Postman

>> [Postman Learning Center](#)

<https://learning.postman.com>

DSA (Continue Daily)

>> [Kunal Kushwaha - DSA Playlist \(Your existing resource\)](#)

https://youtube.com/playlist?list=PL9gnSGHSqcnr_DxHsP7AW9ftq0AtAyYqJ

>> [Striver's A2Z DSA Sheet](#)

<https://takeuforward.org/dsa/strivers-a2z-sheet-learn-dsa-a-to-z>

>> [Take U Forward YouTube - Striver DSA Videos](#)

<https://www.youtube.com/@takeUforward>

By Day 10, You Will Have

- Advanced Java (Streams, Lambdas, Collections) - Learned
- Spring Boot + REST APIs - Learned + Built
- Spring Data JPA + Hibernate + MySQL - Learned + Built
- Spring Security + JWT Auth - Learned + Built
- React.js (Hooks, Router, Axios) - Learned + Built
- Full-Stack Integration (React + Spring Boot) - Built
- Project 1: Employee Management API (Backend) - Complete
- Project 2: Full-Stack Employee Management (React + Spring Boot) - Complete
- Project 3: Blog/Notes App (Speed Build) - Complete
- Deployed projects (live URLs) - Deployed
- Updated resume + GitHub - Ready
- 20-30 DSA problems solved - Practiced

CAUTION

10 days is intense. Don't aim for perfection -- aim for working code + understanding. You can always deepen your knowledge later. The goal is to build momentum and a portfolio.

TIP

Come to Antigravity anytime you're stuck -- whether it's a bug, a concept, or project guidance!